UCTMA Teknik Dokümantasyonu

Güncel mimari, API, güvenlik, kurulum ve işletim kılavuzu

2026-10-19

İçindekiler

UCTMA Teknik Dokümantasyonu 6
1. Dokümanın amacı 6
2. Sistem özeti 6
3. Bileşenler 6
3.0 Projeyi Docker ile başlatma 6
3.1 Backend 7
3.2 Frontend 7
3.3 Docling Converter Service 8
3.4 RabbitMQ 8
3.5 PostgreSQL 9
4. Docker network ve volume yapısı 10
5. Ortam değişkenleri 10
5.1 Backend ortam değişkenleri 10
5.2 Converter ortam değişkenleri 11
5.3 Frontend ortam değişkeni 12
6. Backend database modeli 12
6.1 users tablosu 12
6.2 files tablosu 12
6.3 file_chunks tablosu 13
7. Authentication ve Authorization 13
8. User işlemleri 14
8.1 İlk admin hesabı 14
8.2 User oluşturma 14
8.3 Login 15

8.4 İlk şifre değişikliği . . . 15
8.5 User silme . . . 15
9. File işlemleri . . . 15
9.1 Desteklenen uzantılar . . . 15
9.2 File upload . . . 16
9.2.1 URL'den dönüştürme . . . 17
9.3 File listeleme . . . 17
9.4 Yeniden dönüştürme . . . 18
9.5 File silme . . . 18
9.6 Orijinal file indirme . . . 18
9.7 Markdown indirme . . . 18
10. Chunk işlemleri . . . 19
10.1 Chunk üretimi . . . 19
10.2 Chunk listeleme . . . 19
10.3 Tek chunk indirme . . . 19
10.4 Tüm chunk'ları ZIP olarak indirme . . . 19
10.5 Chunk silme . . . 19
11. RabbitMQ mesaj formatı . . . 19
12. Converter işleme davranışı . . . 20
12.1 Docling dosyaları . . . 20
12.2 Plain text dosyaları . . . 20
12.3 Dosya yoksa . . . 20
12.4 Desteklenmeyen uzantı . . . 20
12.5 URL kaynağı . . . 21
13. Callback iş akışı . . . 21
14. Consumer acknowledgement davranışı . . . 22

15. File status davranışı . . . 22
16. Frontend mimarisi . . . 22
16.1 Theme . . . 23
16.2 Frontend API davranışı . . . 23
17. REST API özeti . . . 23
17.1 Authentication endpoint'leri . . . 23
17.2 File endpoint'leri . . . 23
18. Hata yönetimi . . . 24
18.1 401 callback hatası . . . 24
18.2 File PENDING kalırsa . . . 24
18.3 Desteklenmeyen format . . . 25
18.4 Token length uyarısı . . . 25
18.5 URL indirme hataları . . . 25
19. Kurulum ve çalıştırma . . . 25
19.1 Ön koşullar . . . 25
19.2 Backend'i başlatma . . . 26
19.3 Converter'ı başlatma . . . 26
19.4 Frontend'i başlatma . . . 26
20. Doğrulama kontrol listesi . . . 27
21. FastAPI endpoint'leri . . . 27
22. Güvenlik ve işletim notları . . . 28
23. Mevcut sürüm kapsamı . . . 28
24. Mevcut sürümde bulunmayan işlevler . . . 29
25. Bakım kuralları . . . 29
26. URL'den Markdown'a dönüştürme . . . 29
26.1 Kullanıcı akışı . . . 29

26.2 Veri ve sınırlar 30

26.3 İşletim 30

UCTMA Teknik Dokümantasyonu

1. Dokümanın amacı

Bu doküman UCTMA projesinin mevcut mimarisini, özelliklerini ve çalıştırma adımlarını açıklar. URL'den Markdown'a dönüştürme özelliği de mevcut sürüme dahildir.

Doküman aşağıdaki bileşenleri kapsar:

- Backend
- Frontend
- Docling Converter Service
- RabbitMQ
- PostgreSQL
- Docker Compose
- Dosya depolama
- Authentication ve Authorization
- API endpoint'leri
- Hata yönetimi
- Kurulum ve bakım

Bu dokümanda kullanılan bilgiler proje kaynak kodundan ve Compose dosyalarından alınmıştır.

2. Sistem özeti

UCTMA, desteklenen dosyaları Markdown formatına dönüştürür.

İsteğe bağlı olarak Markdown içeriğini chunk olarak da üretir.

Dosya işleme asenkron yapılır.

İş akışı aşağıdaki gibidir:

1. Kullanıcı Frontend üzerinden login olur.
2. Kullanıcı bilgisayarından bir dosya seçer veya herkese açık bir HTTP/HTTPS URL'si girer.
3. Frontend dosyayı Backend'e yükler veya URL'yi Backend'e JSON olarak gönderir.
4. Backend yüklenen dosyayı `/uploads` altında saklar; URL kayıtlarında `filePath` alanında URL'yi saklar.
5. Backend kaynak kaydını PostgreSQL'e ekler.
6. Backend RabbitMQ'ya bir iş mesajı gönderir.
7. Docling Converter Service mesajı RabbitMQ'dan alır.
8. Converter dosyayı ortak `/uploads` volume'ünden okur veya URL sayfasını HTTP(S) üzerinden indirir.
9. Converter kaynağı Markdown'a dönüştürür.
10. Converter, istenmişse chunk üretir.
11. Converter sonucu Backend callback endpoint'ine gönderir.
12. Backend Markdown ve chunk kayıtlarını saklar.
13. Frontend dosya listesini yeniler.

3. Bileşenler

3.0 Projeyi Docker ile başlatma

Linux veya macOS'ta Docker ve Docker Compose kurulduktan sonra proje kök dizininden aşağıdaki komut çalıştırılabilir:

bash

```
./start.sh
```

İlk çalıştırmada script, backend, converter ve frontend için eksik `.env` dosyalarını ilgili `.env.example` şablonlarından oluşturur; JWT anahtarı, ilk admin parolası ve backend/converter arasında ortak callback sırrı üretir. İlk admin bilgileri `uctma/.env` dosyasına yazılır; callback sırrı iki servisin `.env` dosyasında eşit tutulur. Ardından PostgreSQL/RabbitMQ, backend, model prefetch, Docling converter ve frontend servislerini başlatır. Model volume'ü önceden yoksa oluşturulur. İlk Docker build'i ve model indirmeleri internet bağlantısı gerektirebilir. Prefetch başarısız olsa da başlatma devam eder; ilk dönüştürme isteğinde eksik modelleri indirmek için converter internet erişimini kullanır. İnternetsiz kullanım öncesinde gerekli modellerin prefetch edilmesi gerekir.

Compose ile host'a yayımlanan servis portları yalnızca `127.0.0.1` arayüzüne bağlanır; servisler birbirleriyle `uctma_default` Docker ağı üzerinden iletişim kurar. Arayüz `http://localhost:3000` adresindedir.

3.1 Backend

Backend aşağıdaki teknolojileri kullanır:

- Java 21
- Spring Boot `4.2.0-M2`
- Spring Web MVC
- Spring Data JPA
- Spring Security
- Spring Validation
- Spring AMQP RabbitMQ
- PostgreSQL Driver
- JJWT
- Springdoc OpenAPI
- Spring Mail

Backend'in kaynak dizini:

text

```
uctma/src/main/java/com/app/uctma/backend
```

Backend'in container içindeki HTTP portu `8080`'dir.

Host'taki `8080` portu yalnızca `127.0.0.1` arayüzüne yayımlanır.

Backend Docker image'i Maven ile build edilir.

Dockerfile, Maven build aşamasında Java 21 kullanır.

3.2 Frontend

Frontend aşağıdaki teknolojileri kullanır:

- Node.js 20 Alpine image
- Next.js `14.2.15`
- React `18.3.1`
- React DOM `18.3.1`
- `react-markdown` ve `remark-gfm` ile güvenli Markdown/GFM render
- TypeScript `5.7.2`
- Tailwind CSS `3.4.16`

Frontend container portu `3000`'dür.

Host'taki frontend portu yalnızca `127.0.0.1` arayüzüne yayımlanır.

Frontend, Backend'e doğrudan tarayıcı isteği göndermez.

Frontend `/api/*` isteklerini Next.js `rewrite` kuralı ile Backend'e yönlendirir.

Rewrite hedefi aşağıdaki biçimdedir:

text

```
${BACKEND_URL}/uctma/api/*
```

3.3 Docling Converter Service

Converter Python `3.11-slim` image'i üzerinde çalışır.

Converter container aynı anda iki işlem başlatır:

1. FastAPI HTTP server
2. RabbitMQ consumer

FastAPI server `8000` portunda çalışır.

Host'taki `8000` portu yalnızca `127.0.0.1` arayüzüne yayımlanır.

RabbitMQ consumer `file_processing_queue` kuyruğunu dinler.

Converter Dockerfile'daki başlangıç komutu aşağıdaki iki işlemi paralel başlatır:

text

```
uvicorn main:app --host 0.0.0.0 --port 8000
python consumer.py
```

Converter model cache için Docker volume `docling_converter_service_docling_models`'ü `/app/models_cache` dizinine bağlar. Hugging Face ve Whisper cache'leri (`HF_HOME` ve `XDG_CACHE_HOME`) bu kalıcı dizini kullanır. `model-prefetch` servisi Docling'in varsayılan modellerini, RapidOCR modellerini ve kullanılan Whisper Base modelini önceden indirir:

bash

```
cd uctma
docker compose run --rm model-prefetch
```

PDF/görsel isteğinde RapidOCR model dosyaları belirtilen konumda yoksa converter bunları indirmeyi tekrar dener ve aynı kalıcı volume'e yazar. Diğer Docling modelleri cache'te yoksa Docling bunları kullanım sırasında indirmeyi dener. Bu nedenle internetsiz çalıştırmadan önce gerekli modeller prefetch edilmeli ve indirme işleminin başarılı olduğu doğrulanmalıdır. Volume, `docker compose down -v` veya ilgili volume'un silinmesiyle kaldırılır.

3.4 RabbitMQ

RabbitMQ şu image ile çalışır:

text

```
rabbitmq:3-management-alpine
```

RabbitMQ container adı:

text

```
uctma_rabbitmq
```

RabbitMQ AMQP portu `5672`'dir.

RabbitMQ Management portu `15672`'dır.

AMQP ve Management portları host'ta yalnızca `127.0.0.1` arayüzüne yayımlanır.

RabbitMQ verileri `rabbitmq_data` volume'ünde tutulur.

Backend aşağıdaki kaynakları tanımlar:

- Queue: `file_processing_queue`
- Exchange: `file_processing_exchange`
- Routing key: `file_processing_routing_key`

Queue durable olarak tanımlanır.

Exchange `DirectExchange` olarak tanımlanır.

Backend queue ile exchange arasında binding oluşturur.

Converter consumer queue'yu durable olarak tanımlar.

Converter `prefetch_count=1` kullanır.

Bu ayar aynı anda bir mesaj işlenmesini sağlar.

3.5 PostgreSQL

PostgreSQL şu image ile çalışır:

text

```
postgres:15-alpine
```

PostgreSQL container adı:

text

```
rag-platform-postgres
```

PostgreSQL database adı `.env` içindeki `DB_NAME` değerinden alınır.

Mevcut yapılandırmada database adı `rag_database`'dir.

PostgreSQL kullanıcı adı `.env` içindeki `DB_USERNAME` değerinden alınır.

Mevcut yapılandırmada kullanıcı adı `rag_admin`'dir.

PostgreSQL container portu `5432`'dir.

Host portu `DB_HOST_PORT` ile belirlenir ve yalnızca `127.0.0.1` arayüzüne yayımlanır.

Mevcut yapılandırmada host portu `5433`'tür.

PostgreSQL verileri `pgdata` volume'ünde tutulur.

Backend PostgreSQL'e container network üzerinden aşağıdaki biçimde bağlanır:

text

```
jdbc:postgresql://postgres-db:${DB_CONTAINER_PORT}/${DB_NAME}
```

Backend'te şu ayar kullanılır:

text

```
spring.jpa.hibernate.ddl-auto=validate
```

Hibernate database şemasını değiştirmez.

Database değişikliklerini Flyway migration'ları yönetir.

Flyway migration dosyaları şu dizindedir:

text

```
uctma/src/main/resources/db/migration
```

Flyway mevcut database için `baseline-on-migrate=true` ve `baseline-version=1` kullanır.

Yeni database için `V1__create_initial_schema.sql` çalışır.

`V2__remove_file_chunk_content.sql`, `file_chunks.content` kolonunu kaldırır.

`V3__expand_file_path_for_urls.sql`, `files.file_path` kolonunu `TEXT` türüne genişletir; URL kaynakları 255 karakteri aşabildiğinden bu migration gereklidir.

`V4__add_file_error_message.sql`, dönüştürme hatasının kullanıcıya gösterilebilmesi için `files.error_message` kolonunu ekler.

Mevcut database için Flyway version `1` olarak baseline eder ve henüz uygulanmamış sonraki migration'ları sırayla çalıştırır. Backend başlatıldığında migration'lar otomatik uygulanır; Docker image build'i tek başına migration çalıştırmaz.

4. Docker network ve volume yapısı

Backend Compose ve Converter Compose aşağıdaki external network adını kullanır:

```text
uctma_default
```

Frontend Compose bu network'ü external olarak kullanır.

Backend ve Converter aşağıdaki ortak volume'ü kullanır:

```text
uctma_uploads
```

Backend bu volume'ü `/uploads` olarak mount eder.

Converter bu volume'ü `/uploads` olarak mount eder.

Bu ortak volume aşağıdaki dosyaları içerir:

- Orijinal yüklenen dosyalar.
- Üretilen Markdown dosyaları.
- Üretilen chunk dosyaları.

Converter ayrıca model cache için şu volume'ü kullanır:

```text
docling_converter_service_docling_models
```

PostgreSQL volume'ü:

```text
pgdata
```

RabbitMQ volume'ü:

```text
rabbitmq_data
```

5. Ortam değişkenleri

5.1 Backend ortam değişkenleri

Backend `uctma/.env` dosyasındaki değerleri kullanır.

Zip veya temiz bir checkout'ta gerçek `.env` dosyaları bulunmayabilir. `./start.sh` gerekli `.env` dosyalarını `.env.example` şablonlarından üretir; elle başlatmada ilgili örnek dosyaları kopyalayıp ayarları kontrol edin.

Başlıca değişkenler:

Değişken	Kullanım
`DB_NAME`	PostgreSQL database adı
`DB_USERNAME`	PostgreSQL kullanıcı adı

Değişken	Kullanım
DB_PASSWORD	PostgreSQL şifresi
DB_CONTAINER_PORT	PostgreSQL container portu
DB_HOST_PORT	Yalnızca localhost'a yayımlanan PostgreSQL host portu
JWT_SECRET	JWT imzalama anahtarı
JWT_EXPIRATION	JWT geçerlilik süresi
CALLBACK_SHARED_SECRET	Converter callback doğrulaması için paylaşılan sır
BOOTSTRAP_ADMIN_EMAIL	İlk çalıştırmada oluşturulacak admin e-posta adresi
BOOTSTRAP_ADMIN_PASSWORD	İlk admin parolası; `start.sh` tarafından üretilir
SMTP_USERNAME	E-posta hesabı
SMTP_PASSWORD	E-posta hesabı şifresi
SMTP_HOST	SMTP sunucusu
SMTP_PORT	SMTP portu
UPLOAD_PATH	Dosya saklama dizini
RABBITMQ_HOST	RabbitMQ host adı
RABBITMQ_PORT	RabbitMQ AMQP portu
RABBITMQ_USER	RabbitMQ kullanıcı adı
RABBITMQ_PASS	RabbitMQ şifresi
RABBITMQ_QUEUE_FILE	Dosya işleme queue adı
RABBITMQ_EXCHANGE_FILE	Dosya işleme exchange adı
RABBITMQ_ROUTING_KEY_FILE	Dosya işleme routing key değeri

Secret değerleri bu dokümana yazılmamalıdır.

Üretim ortamında varsayılan `guest` bilgileri kullanılmamalıdır.

`start.sh` callback sırrını ilk çalıştırmada üretir ve aynı değeri backend ile converter `.env` dosyalarına yazar. İki servis en az 32 karakterli aynı değeri kullanmalıdır. İlk admin hesabı yalnızca belirtilen e-posta henüz kayıtlı değilse oluşturulur; parola BCrypt ile saklanır ve ilk girişte değiştirilmesi istenir. Üretilen ilk admin parolası `uctma/.env` dosyasında bulunur.

5.2 Converter ortam değişkenleri

Converter `docling_converter_service/.env` dosyasını kullanır.

Başlıca değişkenler:

Değişken	Kullanım
RABBITMQ_HOST	RabbitMQ host adı
RABBITMQ_PORT	RabbitMQ AMQP portu
RABBITMQ_USER	RabbitMQ kullanıcı adı
RABBITMQ_PASS	RabbitMQ şifresi
RABBITMQ_QUEUE_FILE	Dinlenecek queue adı
RABBITMQ_EXCHANGE_FILE	Ortam uyumluluğu için exchange adı
RABBITMQ_ROUTING_KEY_FILE	Ortam uyumluluğu için routing key değeri
BACKEND_CALLBACK_URL	Backend callback adresi
CALLBACK_SHARED_SECRET	Callback isteğinde gönderilen ortak sır

Mevcut callback adresi:

text

```
http://rag-platform-backend:8080/uctma/api/file/callback
```

5.3 Frontend ortam değişkeni

Frontend `BACKEND_URL` değişkenini kullanır.

Mevcut değer:

text

```
http://backend-service:8080
```

Frontend rewrite bu değerin sonuna `/uctma/api` ekler.

Frontend, tamamlanmış dosyaların Markdown önizlemesini de gösterir.

6. Backend database modeli

6.1 `users` tablosu

`User` entity'si `users` tablosuna karşılık gelir.

Alanlar:

- `id`
- `email`
- `password`
- `role`
- `newPasswordFlag`

Bir user birden fazla `FileMetadata` kaydına sahip olabilir.

`email` alanı unique'tir.

6.2 `files` tablosu

`FileMetadata` entity'si `files` tablosuna karşılık gelir.

Alanlar:

- `id`
- `originalFileName`
- `filePath`
- `mdFilePath`
- `fileType`
- `status`
- `error_message`
- `maxTokens`
- `user_id`
- `createdAt`

Bir file bir user'a bağlıdır.

`fileType` bir yüklenen dosyanın MIME türünü, URL kaynağında ise `URL` değerini tutar. URL kayıtlarında `filePath` dosya sistemi yolu değil, kaynak URL'dir. Bu kolon `V3` migration'ı ile `TEXT` olarak saklanır.

Bir file birden fazla `FileChunk` kaydına sahip olabilir.

6.3 `file_chunks` tablosu

`FileChunk` entity'si `file_chunks` tablosuna karşılık gelir.

Alanlar:

- `id`
- `chunkIndex`
- `chunkName`
- `tokenCount`
- `file_id`
- `createdAt`
- `filePath`

Chunk kaydı bir file'a bağlıdır.

File silindiğinde entity cascade ayarı database ilişkisini de etkiler.

Backend ayrıca chunk dosyasını fiziksel olarak siler.

Chunk text değeri yalnızca `/uploads/chunks` altındaki fiziksel dosyada tutulur.

`file_chunks.content` kolonu kullanılmaz.

Bu değişiklik Flyway tarafından aşağıdaki migration ile uygulanır:

sql

```
ALTER TABLE file_chunks
DROP COLUMN IF EXISTS content;
```

SQL dosyası:

text

```
uctma/src/main/resources/db/migration/V2__remove_file_chunk_content.sql
```

Mevcut database için uygulama başladığında Flyway database'i version `1` olarak baseline eder ve ardından henüz uygulanmamış `V2` ve sonraki migration'ları çalıştırır.

Migration öncesinde database yedeği alınmalıdır.

`files.error_message` kolonu `V4__add_file_error_message.sql` ile eklenir ve FAILED callback'inde Converter'ın gönderdiği hata açıklamasını saklar.

7. Authentication ve Authorization

Backend stateless Authentication kullanır.

Authentication JWT ile yapılır.

Session database üzerinde tutulmaz.

Korumalı isteklerde aşağıdaki header gönderilir:

http

```
Authorization: Bearer <JWT>
```

Desteklenen role değerleri:

- `ROLE_USER`
- `ROLE_ADMIN`

`ROLE_ADMIN` aşağıdaki endpoint'lere erişebilir:

- User register
- User list

- User delete

Giriş yapmış her kullanıcı aşağıdaki işlemleri yapabilir:

- Password change
- Kendi file kayıtlarını listeleme
- Kendi file'ını yükleme
- Kendi file'ını silme
- Kendi file'ını yeniden dönüştürme
- Kendi file'ının chunk kayıtlarını listeleme
- Kendi file'ının çıktılarını indirme
- Kendi chunk kaydını silme

`CheckAccess` sınıfı file ve chunk sahipliğini kontrol eder.

Bir kullanıcı başka kullanıcıya ait file veya chunk'a erişemez.

Login endpoint'i public'tir. Callback endpoint'i JWT filtresinden muaf olsa da kimlik doğrulamasız değildir: backend ve converter arasında paylaşılan `CALLBACK_SHARED_SECRET`, `X-UCTMA-Callback-Secret` HTTP header'ı ile doğrulanır. Eksik veya yanlış sır `401 Unauthorized` döndürür.

Swagger endpoint'leri de public olarak yapılandırılmıştır:

- `/v3/api-docs/**`
- `/swagger-ui/**`
- `/swagger-ui.html`

8. User işlemleri

8.1 İlk admin hesabı

`AdminBootstrap`, uygulama açılışında `BOOTSTRAP_ADMIN_EMAIL` ve `BOOTSTRAP_ADMIN_PASSWORD` ayarlarını kontrol eder. İkisi de tanımlı değilse bootstrap atlanır; yalnızca biri tanımlıysa veya parola 12 karakterden kısaysa uygulama başlatılmaz. E-posta adresi henüz kayıtlı değilse BCrypt ile şifrelenmiş, `ROLE_ADMIN` rolünde ve ilk girişte parola değişikliği zorunlu olan hesap oluşturulur. Aynı e-posta mevcutsa hesap değiştirilmez.

`start.sh` ilk admin parolasını rastgele üretip `uctma/.env` dosyasına yazar. İlk giriş için e-posta ve parolayı bu dosyadan alın; parolayı güvenli biçimde saklayın ve ilk girişte değiştirin. `.env` dosyası kaynak kontrolüne eklenmemelidir.

8.2 User oluşturma

Admin aşağıdaki endpoint'i çağırır:

http

```
POST /uctma/api/auth/register
```

Request body:

json

```
{
  "email": "user@example.com",
  "role": "ROLE_USER"
}
```

Backend geçici bir şifre üretir.

Backend şifreyi encode ederek database'e kaydeder.

Backend geçici şifreyi e-posta ile gönderir.

Yeni user için `newPasswordFlag=true` olur.

8.3 Login

Endpoint:

http

```
POST /uctma/api/auth/login
```

Login response aşağıdaki alanları içerir:

- `email`
- `token`
- `tokenType`
- `newPasswordRequired`
- `role`

Frontend token ve role değerlerini `localStorage` içinde tutar.

Kullanılan key'ler:

- `uctma_token`
- `uctma_role`

8.4 İlk şifre değişikliği

Geçici şifre ile login yapıldığında `newPasswordRequired=true` olabilir.

Frontend bu durumda password formunu açar.

Endpoint:

http

```
POST /uctma/api/auth/change-password
```

Request body:

json

```
{
  "currentPassword": "geçici-şifre",
  "newPassword": "yeni-şifre",
  "confirmNewPassword": "yeni-şifre"
}
```

Başarılı işlemden sonra `newPasswordFlag=false` olur.

8.5 User silme

Endpoint:

http

```
DELETE /uctma/api/auth/delete?email=user@example.com
```

Aktif admin kendi hesabını silemez.

9. File işlemleri

9.1 Desteklenen uzantılar

Backend ve Converter aşağıdaki uzantıları desteklenen uzantı olarak tanımlar:

- `.pdf`
- `.docx`
- `.pptx`
- `.html`
- `.csv`

- `.xlsx`
- `.txt`
- `.md`
- Ses: `.wav`, `.mp3`, `.m4a`, `.aac`, `.ogg`, `.flac`
- Video: `.mp4`, `.avi`, `.mov`, `.mkv`, `.webm`
- Görsel: `.png`, `.jpg`, `.jpeg`, `.tif`, `.tiff`, `.bmp`, `.webp`

Backend dosya adının son uzantısını kontrol eder.

Örnek:

text

```
report.pdf
report.py.pdf
```

İki dosyada da son uzantı `.pdf` olarak değerlendirilir.

Dosya içeriği uzantı ile uyumlu olmalıdır.

Bir Python dosyasının adını `.pdf` yapmak dosyayı PDF'e dönüştürmez.

Dosya seçme alanı desteklenen uzantıları filtre olarak sunar. Asıl uzantı doğrulaması upload sırasında Backend tarafından yapılır.

Backend de aynı uzantı kontrolünü tekrar yapar.

9.2 File upload

Endpoint:

http

```
POST /uctma/api/file/upload
```

İstek `multipart/form-data` olmalıdır.

Form alanları:

- `file`
- `conversionType`
- `maxTokens`

`conversionType` değerleri:

- `CONVERT`
- `CONVERT_AND_CHUNK`

Varsayılan `conversionType` değeri `CONVERT`'dür.

Varsayılan `maxTokens` değeri `400`'dür.

Spring Multipart limitleri:

- Maksimum file size: `50MB`
- Maksimum request size: `50MB`

Backend upload sırasında aşağıdaki işlemleri yapar:

1. Login olmuş user'ı bulur.
2. Dosya uzantısını kontrol eder.
3. Dosyayı `UPLOAD_PATH` altında saklar.
4. `PENDING` status ile file metadata oluşturur.
5. RabbitMQ'ya mesaj gönderir.
6. File bilgilerini response olarak döndürür.

Upload response alanları:

- `id`
- `originalFileName`
- `fileType`
- `status`
- `errorMessage`
- `maxTokens`
- `createdAt`

9.2.1 URL'den dönüştürme

Navbar'daki URL'den yükle işlemi, kullanıcıdan bir HTTP/HTTPS adresi alır ve aynı `CONVERT` veya `CONVERT_AND_CHUNK` seçeneklerini kullanır.

Backend endpoint'i:

http

```
POST /uctma/api/file/url
Content-Type: application/json
```

Örnek istek:

json

```
{
  "url": "https://example.com/page",
  "conversionType": "CONVERT",
  "maxTokens": 400
}
```

Backend yalnızca HTTP/HTTPS şemalarını, host içeren ve kullanıcı bilgisi taşımayan URL'leri kabul eder; standart dışı portları reddeder. URL uzunluğu 2048 karakterle sınırlıdır. Kaynak dosya oluşturulmadığı için `filePath` doğrudan URL, `fileType` ise `URL` olarak kaydedilir. URL kaydı da normal dosya gibi kullanıcıya ait olur; listelenebilir, yeniden dönüştürülebilir ve silindiğinde türetilmiş Markdown/chunk dosyaları temizlenir.

Converter URL'yi işlerken HTTP yönlendirmelerini en fazla 5 adım takip eder. Her hedefi tekrar doğrular ve loopback/özel ağ gibi global olmayan IP adreslerini reddeder. Yalnızca `text/html` ve `application/xhtml+xml` içerik türleri kabul edilir; gövde boyutu 10 MB ile sınırlıdır. İndirilen HTML geçici `.html` dosyasına yazılarak mevcut Docling dönüşüm akışına verilir ve geçici dosya işlem sonunda silinir. Sayfalar JavaScript çalıştırılmaz; yalnızca sunucunun döndürdüğü HTML işlenir.

Üçüncü taraf siteler erişimi engelleyebilir, oturum açma isteyebilir veya geçerli TLS sertifika zinciri sunmayabilir. Bu durumlarda dönüşüm `FAILED` olur; TLS doğrulaması kapatılmamalıdır.

9.3 File listeleme

Endpoint:

http

```
GET /uctma/api/file
```

Endpoint yalnızca Authentication yapan user'ın file kayıtlarını döndürür.

Response içinde aşağıdaki alanlar bulunur:

- `id`
- `originalFileName`
- `fileType`
- `status`
- `errorMessage`
- `maxTokens`

- `createdAt`

9.4 Yeniden dönüştürme

Endpoint:

http

```
POST /uctma/api/file/{fileId}/convert
```

Query parametreleri:

- `conversionType`
- `maxTokens`

`conversionType` gönderilmezse `CONVERT` kullanılır.

`maxTokens` gönderilmezse mevcut file değerinin kullanılması beklenir.

Backend yeniden dönüştürme sırasında:

1. File sahipliğini kontrol eder.
2. Orijinal file uzantısını tekrar kontrol eder.
3. Eski chunk dosyalarını siler.
4. Eski chunk database kayıtlarını siler.
5. Eski Markdown dosyasını siler.
6. `mdFilePath` değerini temizler.
7. File status değerini `PENDING` yapar.
8. Varsa önceki `errorMessage` değerini temizler.
9. Yeni RabbitMQ mesajı gönderir.

Orijinal file silinmez.

9.5 File silme

Endpoint:

http

```
DELETE /uctma/api/file/{fileId}
```

Backend aşağıdaki verileri siler:

- Chunk fiziksel dosyaları
- Chunk database kayıtları
- Markdown fiziksel dosyası
- Orijinal fiziksel dosya
- File metadata kaydı

URL kaydında yerel bir orijinal dosya bulunmadığından URL dosya yolu fiziksel olarak silinmez.

9.6 Orijinal file indirme

Endpoint:

http

```
GET /uctma/api/file/{fileId}/download
```

Bu endpoint original file'ı binary response olarak döndürür.

9.7 Markdown indirme

Endpoint:

http

```
GET /uctma/api/file/{fileId}/download-md
```

Markdown dosyası henüz oluşturulmamışsa Backend `mdFilePath` değerini boş bulur.

Bu durumda indirme işlemi için kullanılabilir bir Resource bulunmayabilir.

Frontend bu endpoint'i tamamlanmış output için kullanmalıdır.

10. Chunk işlemleri

10.1 Chunk üretimi

Chunk üretimi yalnızca `CONVERT_AND_CHUNK` seçildiğinde istenir.

Docling belge modeline dönüştürülen tüm girdilerde `HybridChunker` kullanılır.

`HybridChunker` için `maxTokens` değeri kullanılır.

Sayfa bilgisi bulunmayan ses/video gibi girdilerin `pageNumbers` listesi boş olabilir.

10.2 Chunk listeleme

Endpoint:

http

```
GET /uctma/api/file/{fileId}/chunks
```

Response her chunk için şu alanları döndürür:

- `id`
- `chunkName`

Chunk kayıtları `chunkIndex` sırasına göre listelenir.

10.3 Tek chunk indirme

Endpoint:

http

```
GET /uctma/api/file/chunks/{chunkId}/download
```

Response `text/plain` olarak döndürülür.

10.4 Tüm chunk'ları ZIP olarak indirme

Endpoint:

http

```
GET /uctma/api/file/chunks/{fileId}/download-all
```

Response `application/zip` olarak döndürülür.

10.5 Chunk silme

Endpoint:

http

```
DELETE /uctma/api/file/chunks/{chunkId}
```

Backend önce chunk sahipliğini kontrol eder.

Sonra fiziksel chunk dosyasını ve database kaydını siler.

11. RabbitMQ mesaj formatı

Backend'in gönderdiği mesaj `FileMessageDTO` alanlarını kullanır:

json

```
{
  "fileId": 10,
  "filePath": "/uploads/example.pdf",
  "originalFileName": "example.pdf",
  "fileType": "application/pdf",
  "conversionType": "CONVERT_AND_CHUNK",
  "maxTokens": 400
}
```

Converter bu alanları kullanır:

- `fileId`
- `filePath`
- `conversionType`
- `maxTokens`

`fileType` değeri `URL` ise `filePath` URL olarak değerlendirilir; diğer mesajlarda Converter `filePath` ile ortak `/uploads` volume'ündeki dosyayı okur.

Consumer `prefetch_count=1` ile çalışır.

12. Converter işleme davranışı

12.1 Docling dosyaları

`.pdf`, `.docx`, `.pptx`, `.html`, `.xlsx`, görseller, ses ve video Docling'e gönderilir.

Docling conversion sonucundan Markdown export edilir.

PDF ve görsel OCR için RapidOCR'nin `torch` backend'i kullanılır. Gerekli RapidOCR dosyaları `/app/models_cache/RapidOcr` altında aranır; eksik dosyalar çevrimiçi ilk kullanımda bu kalıcı cache'e indirilir.

Ses işleme Whisper Base modelini, video ise Docling'in video pipeline'ını kullanır.

`CONVERT` işleminde chunk üretilmez.

`CONVERT_AND_CHUNK` işleminde `HybridChunker` kullanılır.

Chunk response alanları:

- `chunkIndex`
- `text`
- `pageNumbers`

12.2 Plain text dosyaları

`.csv` Docling'in CSV converter'ından geçirilir ve Markdown olarak export edilir. `.md` içeriği Docling'in Markdown parser'ına verilir. `.txt` içeriği HTML `<pre>` içinde Docling'e aktarılır; dönüştürülen metin olarak orijinal dosya içeriği korunur. `CONVERT_AND_CHUNK` durumunda bu belgeler de aynı `HybridChunker` ile chunk'lanır.

12.3 Dosya yoksa

Converter file path'in varlığını kontrol eder.

File yoksa processing error oluşur.

12.4 Desteklenmeyen uzantı

Converter FastAPI endpoint'leri son uzantıyı kontrol eder.

RabbitMQ consumer doğrudan `process_file_with_docling` fonksiyonunu çağırır.

Backend upload öncesinde desteklenmeyen uzantıyı engeller.

Consumer'a doğrudan desteklenmeyen bir file mesajı gönderilirse processing veya Docling hatası oluşabilir.

Bu hata callback ile `FAILED` status olarak Backend'e bildirilebilir.

12.5 URL kaynağı

RabbitMQ mesajında `fileType: "URL"` olan işler `process_url_with_docling` fonksiyonuna yönlendirilir. Diğer mesajlar dosya işleme fonksiyonunu kullanır. URL için uzantı listesine `URL` eklenmez; URL bir dosya biçimi değil kaynak türüdür. İndirilen içerik geçici `.html` dosyası üzerinden Docling'e verilir.

13. Callback iş akışı

Callback endpoint'i:

http

```
POST /uctma/api/file/callback
```

Bu endpoint Frontend tarafından çağrılmaz.

Bu endpoint yalnızca Converter'ın Backend'e sonucu bildirmesi için kullanılır.

İstek, `X-UCTMA-Callback-Secret` header'ında backend ve converter `.env` dosyalarında ortak olan `CALLBACK_SHARED_SECRET` değerini taşımalıdır. Backend eksik/yanlış değerleri `401 Unauthorized` ile reddeder; sır en az 32 karakter olmalıdır. `start.sh` bu sırrı güvenli rastgele bir değer olarak üretip iki ortam dosyasına eşit yazar. Sır loglara veya kaynak koda eklenmemelidir.

Callback request alanları:

- `fileId`
- `status`
- `convertedText`
- `chunks`
- `errorMessage`

Başarılı callback örneği:

json

```
{
  "fileId": 10,
  "status": "COMPLETED",
  "convertedText": "# Başlık",
  "chunks": [],
  "errorMessage": null
}
```

Hatalı callback örneği:

json

```
{
  "fileId": 10,
  "status": "FAILED",
  "convertedText": null,
  "chunks": [],
  "errorMessage": "Dosya formatı desteklenmiyor"
}
```

Backend callback aldığında:

1. `fileId` ile file metadata kaydını bulur.
2. File status değerini callback status değerine ayarlar.
3. `convertedText` boş değilse Markdown dosyası kaydeder.
4. Chunk listesi boş değilse chunk dosyalarını ve database kayıtlarını oluşturur.
5. Status `FAILED` ise `errorMessage` değerini `files.error_message` alanına kaydeder ve hatayı loglar.

`PROCESSING` veya başarılı callback geldiğinde önceki `errorMessage` değeri temizlenir.

Callback path'i JWT filtresinden muaftır; endpoint bunun yerine paylaşılan callback sırrını sabit-zamanlı karşılaştırmayla doğrular. Converter JWT kullanmadan callback gönderebilir, ancak kimlik doğrulaması

olmadan iş sonucu yazamaz.

14. Consumer acknowledgement davranışı

Converter callback gönderirken aşağıdaki davranışı kullanır:

- Başarılı callback sonrası message `ack` edilir.
- Kalıcı HTTP `4xx` callback hatası sonrası message `ack` edilir.
- Geçici network veya Backend bağlantı hatası sonrası message `nack` ile requeue edilir.

Bu davranış geçersiz bir message'ın sonsuz tekrar işlenmesini engellemeyi amaçlar.

RabbitMQ bağlantısı kapanırsa consumer beş saniye bekler ve yeniden bağlanmayı dener.

RabbitMQ bağlantısında heartbeat değeri varsayılan olarak `600` saniyedir.

15. File status davranışı

Backend enum içinde şu status değerleri vardır:

- `PENDING`
- `PROCESSING`
- `COMPLETED`
- `FAILED`

Upload ve reconvert sırasında Backend status değerini `PENDING` yapar.

RabbitMQ mesajı Converter tarafından alındığında, dönüşüm başlamadan önce `PROCESSING` callback'i gönderilir.

Callback başarılıysa Converter `COMPLETED` gönderir.

Converter işlem hatasında `FAILED` gönderir.

Frontend bu status değerlerini aşağıdaki Türkçe etiketlerle gösterir:

- `Bekliyor`
- `İşleniyor`
- `Tamamlandı`
- `Başarısız`

Frontend file listesini periyodik olarak yeniler.

FAILED satırındaki durum etiketine tıklanınca dolu `errorMessage` açıklaması durumun altında açılır; tekrar tıklamak açıklamayı kapatır.

16. Frontend mimarisi

Frontend dosya yapısı:

text

```
uctma_frontend/
├── app/
│   ├── page.tsx
│   ├── layout.tsx
│   └── globals.css
├── components/
│   ├── ChunksDialog.tsx
│   ├── FileTable.tsx
│   ├── FileUploader.tsx
│   ├── LoginScreen.tsx
│   ├── Modal.tsx
│   ├── Navbar.tsx
│   ├── PasswordForm.tsx
│   ├── RegisterForm.tsx
│   ├── UrlUploader.tsx
```

```
│   └── UsersDialog.tsx
├── hooks/
│   ├── useAuth.ts
│   ├── useFiles.ts
│   └── useTheme.ts
└── lib/
    ├── api.ts
    └── types.ts
```

`app/page.tsx` ekran state'ini ve bileşenler arası akışı yönetir.

`components/` UI bileşenlerini içerir.

`hooks/useAuth.ts` Authentication state'ini yönetir.

`hooks/useFiles.ts` file, chunk ve user API çağrılarını yönetir.

`hooks/useTheme.ts` theme state'ini yönetir.

`lib/api.ts` JSON ve binary API çağrıları için yardımcı fonksiyonlar içerir.

`lib/types.ts` Backend response tiplerini içerir.

16.1 Theme

Tailwind `darkMode: "class"` ayarıyla kullanılır.

Theme seçimi `uctma_theme` localStorage key'i ile saklanır.

Frontend light ve dark theme toggle içerir.

16.2 Frontend API davranışı

JSON API çağrıları `/api` prefix'i ile yapılır.

Korumalı isteklerde `Authorization` header'ı eklenir.

Download isteklerinde response `blob` olarak alınır.

JSON response'larda Backend'in `ApiResponse` zarfı kontrol edilir.

HTTP başarısızsa veya `success=false` ise Frontend error gösterir.

Tamamlanmış dosyaların Markdown içeriği önizleme dialog'unda gösterilebilir. Dosya işlemleri farklı simgelerle sunulur; her simgenin tooltip'i ve erişilebilir adı bulunur.

17. REST API özeti

17.1 Authentication endpoint'leri

Method	Endpoint	Yetki	Açıklama
POST	/uctma/api/auth/login	Public	Login
POST	/uctma/api/auth/register	ROLE_ADMIN	User oluşturma
POST	/uctma/api/auth/change-password	Authenticated	Password değiştirme
GET	/uctma/api/auth/list-users	ROLE_ADMIN	User listesi
DELETE	/uctma/api/auth/delete?email=...	ROLE_ADMIN	User silme

17.2 File endpoint'leri

Method	Endpoint	Yetki	Açıklama
POST	/uctma/api/file/upload	Authenticated	File yükleme

Method	Endpoint	Yetki	Açıklama
POST	/uctma/api/file/url	Authenticated	URL'den Markdown/chunk işleme kuyruğuna ekleme
GET	/uctma/api/file	Authenticated	User file listesi
POST	/uctma/api/file/{fileId}/convert	Authenticated	Yeniden dönüştürme
DELETE	/uctma/api/file/{fileId}	Authenticated	File silme
GET	/uctma/api/file/{fileId}/download	Authenticated	Original file indirme
GET	/uctma/api/file/{fileId}/download-md	Authenticated	Markdown indirme
GET	/uctma/api/file/{fileId}/preview	Authenticated	Markdown önizleme içeriği
GET	/uctma/api/file/{fileId}/chunks	Authenticated	Chunk listesi
DELETE	/uctma/api/file/chunks/{chunkId}	Authenticated	Chunk silme
GET	/uctma/api/file/chunks/{chunkId}/download	Authenticated	Tek chunk indirme
GET	/uctma/api/file/chunks/{fileId}/download-all	Authenticated	ZIP indirme
POST	/uctma/api/file/callback	X-UCTMA-Callback-Secret	Converter işlem sonucu callback

18. Hata yönetimi

Backend `RuntimeException` hatalarını `400 Bad Request` response'una dönüştürür.

Validation hataları da `400 Bad Request` response'u olarak döndürülür.

Backend JSON response zarfında `success`, `data` ve `error` alanlarını kullanır.

Frontend `success=false` response'larını hata olarak işler.

18.1 `401` callback hatası

Callback isteğinde `X-UCTMA-Callback-Secret` header'ı yoksa veya backend ile converter'ın `CALLBACK_SHARED_SECRET` değerleri farklıysa Backend `401` döndürür.

Çalışan container callback isteğine `401` döndürürse aşağıdakiler kontrol edilmelidir:

1. Backend ve converter `.env` dosyalarındaki `CALLBACK_SHARED_SECRET` değerlerinin aynı ve en az 32 karakter olup olmadığı.
2. Her iki container'ın `.env` dosyalarındaki yeni değeri alacak şekilde yeniden oluşturulup oluşturulmadığı.
3. Converter'ın doğru Backend container adına bağlanıp bağlanmadığı.
4. `uctma_default` network'ünün iki Compose projesinde de aynı olup olmadığı.
5. `BACKEND_CALLBACK_URL` değerinin doğru olup olmadığı.

Backend'i yeniden build etmek için:

bash

```
cd uctma
docker compose up -d --build backend-service
```

18.2 File `PENDING` kalırsa

Aşağıdaki loglar kontrol edilmelidir:

bash

```
docker logs --tail 200 rag-platform-backend
docker logs --tail 200 docling_converter_service
```

Kontrol sırası:

1. RabbitMQ consumer'ın çalışıp çalışmadığı.
2. File'ın ortak `/uploads` volume'ünde bulunup bulunmadığı.
3. Converter callback URL'sinin doğru olup olmadığı.
4. Callback HTTP status değerinin `200` olup olmadığı.
5. Backend'in callback'i kabul edip etmediği.

18.3 Desteklenmeyen format

Dosyanın son uzantısı kontrol edilmelidir.

Dosya içeriği ile uzantı aynı formatı göstermelidir.

Sadece file adını değiştirmek format dönüşümü yapmaz.

18.4 Token length uyarısı

Docling veya model katmanında tokenizer limitini aşan içerik uyarısı görülebilir.

Bu mesaj tek başına Backend callback'inin başarısız olduğu anlamına gelmez.

Sonuç status ve callback response birlikte kontrol edilmelidir.

18.5 URL indirme hataları

URL işlemi `FAILED` olursa Converter logundaki hata kaynağı ayırt edilmelidir:

- `403 Forbidden`: Hedef site otomatik istekleri reddediyor veya erişim istiyor.
- `SSLCertVerificationError`: TLS sertifikası ya da ara sertifika zinciri istemcinin CA deposuyla doğrulanamıyor. `verify=False` ile doğrulama kapatılmamalıdır.
- İçerik türü/10 MB/yönlendirme hataları: URL işleyicisinin izin verdiği sınırlar aşılmıştır.

Converter image'inde `ca-certificates` paketi kurulur ve Requests için `REQUESTS_CA_BUNDLE=/etc/ssl/certs/ca-certificates.crt` ayarlanır. Dockerfile değişikliğinden sonra container yeniden build edilmelidir:

bash

```
cd docling_converter_service
docker compose up -d --build
```

Sistem CA paketinin bulunması, hedef sitenin eksik veya yanlış sertifika zincirini onarmaz. Böyle bir durumda site yöneticisinin tam zinciri sunması ya da kurumun resmi CA sertifikasının güvenli biçimde container'a eklenmesi gerekir.

19. Kurulum ve çalıştırma

19.1 Ön koşullar

Gerekli araçlar:

- Docker
- Docker Compose

Java ve Node.js, Docker image'lerinin build aşamalarında kullanılır; host makineye ayrıca kurulumları gerekmez. İlk image build'i ve kullanılmamış modellerin indirilmesi için internet erişimi gerekir.

Backend ve Converter aynı Docker network'üne bağlanmalıdır. Proje kökündeki `./start.sh` Compose adımlarını sıralar ve `.env` dosyalarını örnek şablonlardan oluşturur. SMTP ile e-posta gönderimi

kullanılacaksa `uctma/.env` içindeki SMTP ayarları ayrıca yapılandırılmalıdır.

Compose servislerini elle başlatırken backend ve converter `.env` dosyalarına aynı, en az 32 karakterli `CALLBACK_SHARED_SECRET` değeri yazılmalıdır. `BOOTSTRAP_ADMIN_EMAIL` ve `BOOTSTRAP_ADMIN_PASSWORD` birlikte yapılandırılmalıdır; ikisi de boş bırakılırsa admin bootstrap atlanır. Örnek değerler (`GENERATED_BY_START_SH`) elle başlatma için kullanılabilir secret değildir. Bu ayarları otomatik ve tutarlı oluşturmak için proje kökünden `./start.sh` kullanılması önerilir.

19.2 Backend'i başlatma

bash

```
cd uctma
docker compose up -d --build
```

Bu Compose projesi şu servisleri başlatır:

- `postgres-db`
- `rabbitmq`
- `backend-service`
- `model-prefetch` (tek seferlik model indirmesi; tamamlanınca kapanır)

Backend Compose içinde `backend-service`, `postgres-db` servisine `depends_on` ile bağlıdır.

RabbitMQ için Compose seviyesinde `depends_on` tanımı yoktur. Backend Compose'u elle başlatmadan önce harici model volume'ünü oluşturun:

bash

```
docker volume create docling_converter_service_docling_models
```

İnternetsiz kurulum için model-prefetch tamamlanmış olmalı; ilk kullanımda eksik model indirmesi yapacak internet erişimi bulunmuyorsa dönüştürme başarısız olabilir.

19.3 Converter'ı başlatma

bash

```
cd docling_converter_service
docker compose up -d --build
```

Converter'ın bağlı olduğu `uctma_default` network'ü mevcut olmalıdır.

Converter'ın bağlı olduğu `uctma_uploads` volume'ü Backend ile aynı volume olmalıdır.

19.4 Frontend'i başlatma

bash

```
cd uctma_frontend
docker compose up -d --build
```

Frontend adresi:

text

```
http://localhost:3000
```

Backend host adresi (localhost):

text

```
http://localhost:8080
```

Converter health endpoint'i ve RabbitMQ Management arayüzü de localhost'tan erişilebilir:

text

```
http://localhost:15672
```

PostgreSQL host bağlantı portu mevcut ayarda:

text

```
5433
```

Yayımlanan bu portlar yalnızca `127.0.0.1` arayüzüne bağlanır. Container'lar birbirlerine host portlarını kullanmadan `uctma_default` Docker ağı üzerinden erişir. Bu ağ Internet'e yayımlanmaz; ancak ağa bağlı container'ların kendi aralarındaki erişimi devam eder.

20. Doğrulama kontrol listesi

Kurulumdan sonra aşağıdaki kontroller yapılmalıdır:

1. PostgreSQL container çalışıyor mu?
2. RabbitMQ container çalışıyor mu?
3. RabbitMQ Management arayüzü açılıyor mu?
4. Backend `8080` portunda çalışıyor mu?
5. Converter `8000` portunda çalışıyor mu?
6. Converter `/health` endpoint'i cevap veriyor mu?
7. Frontend `3000` portunda açılıyor mu?
8. Login çalışıyor mu?
9. File upload sonrası queue'da mesaj oluşuyor mu?
10. Converter ortak volume'deki file'ı okuyabiliyor mu?
11. Callback ortak sırla gönderiliyor ve başarılı istekte `200` dönüyor mu?
12. File status `COMPLETED` veya `FAILED` oluyor mu?

Converter health endpoint'i:

http

```
GET http://localhost:8000/health
```

Beklenen response:

json

```
{
  "status": "healthy",
  "service": "Docling Converter Service"
}
```

21. FastAPI endpoint'leri

Converter container ayrıca aşağıdaki HTTP endpoint'lerini sağlar:

Method	Endpoint	Açıklama
GET	/health	Converter sağlık kontrolü
POST	/convert	Bir file'ı Markdown'a dönüştürme
POST	/convert-and-chunk	Bir file'ı Markdown'a dönüştürme ve chunk üretme

`/convert` ve `/convert-and-chunk` endpoint'leri UploadFile kabul eder.

Converter FastAPI endpoint'leri desteklenmeyen son uzantılar için `400` döndürür.

İşleme hataları için `500` döndürür.

RabbitMQ consumer ise bu HTTP endpoint'lerini çağırmaz.

Consumer doğrudan `docling_processor.py` içindeki processing fonksiyonunu çağırır.

22. Güvenlik ve işletim notları

`.env` dosyaları secret içerir.

`.env` dosyaları kaynak kodu ile paylaşılmamalıdır.

JWT secret production ortamında güçlü ve gizli tutulmalıdır.

SMTP password gizli tutulmalıdır.

RabbitMQ için production ortamında `guest/guest` kullanılmamalıdır.

Callback endpoint'i JWT filtresinden muaf olmakla birlikte `X-UCTMA-Callback-Secret` header'ında paylaşılan sırrı zorunlu tutar. Sır, backend ve converter `.env` dosyalarında aynı, en az 32 karakterli değer olmalıdır.

Compose port eşlemeleri host'ta yalnızca `127.0.0.1`'e yayımlanır. Bu, uzaktaki istemcilerin host portlarına doğrudan erişmesini önler; ancak `uctma_default` ağına bağlanmış container'ların birbirlerine erişimini engellemez. Docker ağı kendi başına Internet'e yayımlanmış değildir.

Production ortamında HTTPS kullanılmalıdır.

Swagger endpoint'leri production ortamında erişim politikası ile sınırlandırılmalıdır.

23. Mevcut sürüm kapsamı

Mevcut sürümde bulunan işlevler:

- JWT login
- Role tabanlı Authorization
- Admin user oluşturma
- Admin user listeleme
- Admin user silme
- İlk login password değişimi
- File upload
- Markdown conversion
- Markdown ve chunk conversion
- File status listeleme
- File reconversion
- Tamamlanmış Markdown dosyası için önizleme
- Original file indirme endpoint'i
- Markdown indirme
- Chunk listeleme
- Tek chunk indirme
- Chunk ZIP indirme
- File silme
- Chunk silme
- Light theme
- Dark theme
- Responsive temel UI
- RabbitMQ ile asenkron processing
- PostgreSQL ile kalıcı metadata saklama

24. Mevcut sürümde bulunmayan işlevler

Aşağıdaki işlevler kaynak kodda mevcut bir özellik olarak doğrulanmamıştır:

- Pagination
- File search
- Status filter
- Upload progress bar
- Queue progress bar
- Audit log
- Kullanıcı rolünü sonradan değiştirme
- Otomatik end-to-end test suite
- Otomatik Frontend unit test suite

25. Bakım kuralları

Queue, exchange ve routing key değerleri Backend ve Converter ortamlarında aynı tutulmalıdır.

Backend ve Converter aynı `uctma_default` network'üne bağlı olmalıdır.

Backend ve Converter aynı `uctma_uploads` volume'ünü kullanmalıdır.

Callback request alanları Backend `FileProcessingCallbackReqDTO` ile uyumlu olmalıdır.

Role değerleri aşağıdaki biçimde kullanılmalıdır:

- `ROLE_USER`
- `ROLE_ADMIN`

Conversion değerleri aşağıdaki biçimde kullanılmalıdır:

- `CONVERT`
- `CONVERT_AND_CHUNK`

Kaynak kod değişikliğinden sonra ilgili image yeniden build edilmelidir:

bash

```
docker compose up -d --build
```

Database volume'ü silinirse PostgreSQL verileri silinir.

RabbitMQ volume'ü silinirse RabbitMQ kalıcı verileri silinir.

Uploads volume'ü silinirse original file, Markdown file ve chunk file verileri silinir.

26. URL'den Markdown'a dönüştürme

URL'den dönüştürme, mevcut dosya yükleme ve asenkron dönüştürme akışıyla birlikte çalışır.

26.1 Kullanıcı akışı

1. Kullanıcı giriş yaptıktan sonra navbar'daki URL'den yükle butonuna basar.
2. HTTP/HTTPS sayfa adresini ve dönüşüm türünü seçer.
3. Frontend `POST /uctma/api/file/url` isteğini JWT ile Backend'e gönderir.
4. Backend URL kaynak kaydını `PENDING` durumunda oluşturur ve RabbitMQ'ya `fileType: "URL"` alanlı mesaj bırakır.
5. Consumer URL'yi güvenli indirme kontrollerinden geçirir; HTML'i mevcut Docling pipeline'ına verir.
6. Markdown/chunk sonucu mevcut callback üzerinden Backend'e kaydedilir.

7. URL dosya listesinde URL etiketiyle gösterilir; Markdown indirme, chunk görüntüleme, önizleme, yeniden dönüştürme ve silme mevcut dosya işlemlerini kullanır.

26.2 Veri ve sınırlar

- `files.file_path`: dosya yüklemelerinde yerel yol, URL kaynaklarında tam URL.
- `files.file_type`: yüklemelerde içerik türü, URL kaynaklarında `URL`.
- `filePath` sütunu `V3` Flyway migration'ı ile `TEXT` olur.
- URL uzunluğu en fazla 2048 karakterdir; HTTP/HTTPS dışındaki şemalar ve standart dışı portlar kabul edilmez.
- Worker yalnızca herkese açık HTML sayfalarını işler; özel ağ IP'lerini engeller, yönlendirmeleri sınırlar ve HTML indirmesini 10 MB ile sınırlar.
- Hedef sitenin erişim, robots veya TLS politikaları dönüşümü engelleyebilir; uygulama TLS doğrulamasını devre dışı bırakmaz.

26.3 İşletim

URL desteği için converter bağımlılıklarında `requests` bulunur. Docker image sistem CA sertifikalarını yükler ve Requests'e sistem CA bundle yolunu bildirir. Bu ayar yalnızca container'ın güvenilir CA deposunu kullanmasını sağlar; hedef sunucunun eksik sertifika zincirini düzeltmez. Kod veya Dockerfile değişikliğinden sonra converter image'i yeniden build edilmelidir.